

A. PROJECT SCOPE

- This project focuses on developing an **AI-based flood monitoring system**.
- It collects **real-time sensor data** (e.g., water level, rainfall).
- Data is stored and retrieved from **SQL Server database**.
- The system provides:
 - Flood risk detection
 - Alerts for authorities
 - Safe route suggestions for users
- It ensures **public safety during flood situations**.

B. GOALS AND OBJECTIVES

- **Real-time Monitoring**
 - Live flood data access from database
- **Risk Mitigation**
 - AI-based analysis to detect flood danger
- **Safe Navigation**
 - Suggest alternate safe routes
- **Secure Access**
 - Only authorized users can manage system

C. BASIC FEATURES

- **Sensor Data Dashboard**
 - Shows water level and rainfall status
- **Authority Login Portal**
 - Secure login system for admin
- **Active Alert System**
 - Detects flood risk and shows alerts
- **Dynamic Routing**
 - Helps users avoid flooded areas

D. CODE AND QUERIES

Backend Connection

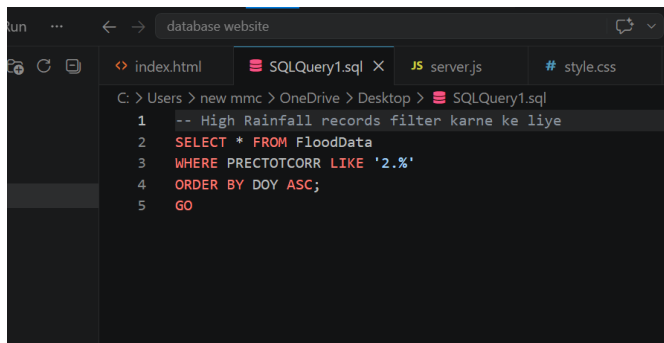
- Backend developed using **Node.js**
- Server runs on:
 - `http://localhost:5000`
- It connects frontend with SQL Server database

Database Query (Example)

☞ Purpose:

- Detect high rainfall areas

- Identify flood risk zones



SQL SERVER DATABASE INTEGRATION & QUERY EXECUTION

CODE: SERVER.JS

- `server.js` is the **main backend file** of the system.
- It is developed using **Node.js** and **Express.js**.
- It establishes a **connection between the application and SQL Server database**.
- The database configuration includes:
 - Server name (localhost)
 - Database name (FloodDB)
 - Username and password
- The server runs on **port 5000**.
- Once connected, it allows execution of SQL queries for flood data.

API ROUTES

- API routes are used to **handle client requests** and send responses.

```

14 // 2. Test Route: Check karne ke liye ke server SQL se connect hua ya nahi
15 app.get('/api/status', async (req, res) => {
16   try {
17     await sql.connect(config);
18     res.json({ status: "Success", message: "Connected successfully to FloodGuard_DB!" });
19   } catch (err) {
20     res.status(500).json({ status: "Failed", message: err.message });
21   }
22 });
23
24 // 3. Main Data Route: Database se Filtered Data frontend ko bhejna
25 app.get('/api/high-risk', async (req, res) => {
26   try {
27     let pool = await sql.connect(config);
28     // Wahi filter query jo aapne SSMS mein test ki thi!
29     let result = await pool.request().query("SELECT TOP 20 * FROM FloodData WHERE PRECOTCORR LIKE '2.2%'");
30     res.json(result.recordset);
31   } catch (err) {
32     res.status(500).send("Database Error: " + err.message);
33   }
34 });

```

SCRIPT.JS

- `script.js` is the **frontend JavaScript file**.
- It is responsible for **communicating with the backend server**.
- It uses the **Fetch API** to send requests and receive data.
- It retrieves flood data from the backend and displays it on the webpage.

FETCH() FUNCTION

- The `fetch()` function is used to **call backend APIs**.
- It sends an HTTP request (GET/POST) to the server.
- It receives data in **JSON format**.
- The response is then converted and displayed on the interface.

```
database website > JS script.js > checkSystemStatus
1  async function fetchLiveFloodData() {
2      try {
3          console.log("Fetching live data from backend...");
4          let response = await fetch('http://localhost:5000/api/high-risk');
5          if (!response.ok) throw new Error("Server error");
6          let data = await response.json();
7
8          console.log("Data Received:", data);
9          document.getElementById('water-level').innerText = "Status: Connected - " + data.length + " Records Active";
10         alert("✅ Success! Connected to SQL Server. Records fetched: " + data.length);
11     } catch (error) {
12         alert("❌ Error: Backend server is offline. Run 'node server.js' first!");
13     }
14 }
15
16 function handleAuthorityLogin(event) {
17     if (event) event.preventDefault();
18     let username = prompt("Enter Authority Username:");
19     if (username) {
20         alert(`Executing SQL Query for: ${username}\n\nAccess Granted!`);
21     }
22 }
23 // Status check karne ka naya function
24 function checkSystemStatus() {
25     alert("SYSTEM STATUS: OK\nDatabase: Connected\nFloodGuard AI: Active (Real-time monitoring)");
26     console.log("System Status Checked: All systems operational.");
27 }
```

INDEX.HTML (THE INTERFACE)

- `index.html` is the **user interface of the system**.
- It contains buttons, headings, and display sections.
- It allows users to interact with the system.
- Data received from backend is shown here.

ONCLICK() CALL

- `onclick()` is used to **trigger a function when a button is clicked**.
- It connects user action with JavaScript function.
- When the button is clicked:

- It calls a function in `script.js`
- That function uses `fetch()` to get data

```

37         <div class="card alerts-card">
38             <h3>Active Alerts</h3>
39             <p>Risk Level: Low</p>
40             <button class="card-btn btn-orange" onclick="checkSystemStatus()">Check Status</butt
41         </div>
42         <div class="card nav-card">
43             <h3>Safe Navigation</h3>
44             <button class="nav-btn-bold" onclick="window.open('https://www.google.com/maps', '_bl
45         </div>
46     </div>
47 </section>
48

```

OVERALL FLOW

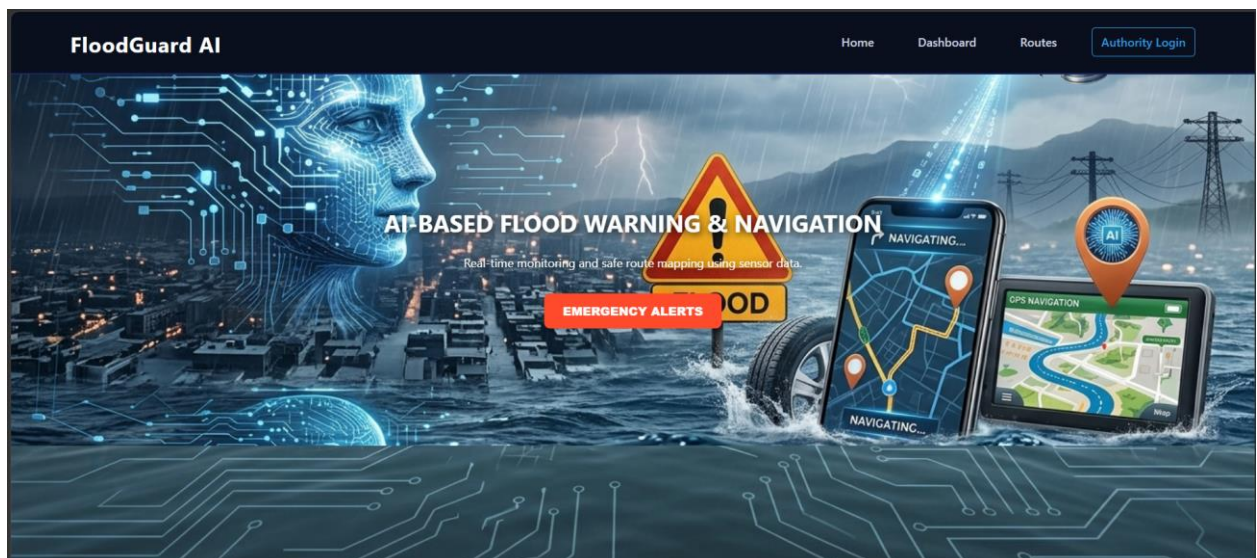
1. User clicks button (onclick)
2. Function is triggered in `script.js`
3. `fetch()` sends request to backend
4. Backend returns data from SQL Server
5. Data is displayed on `index.html`

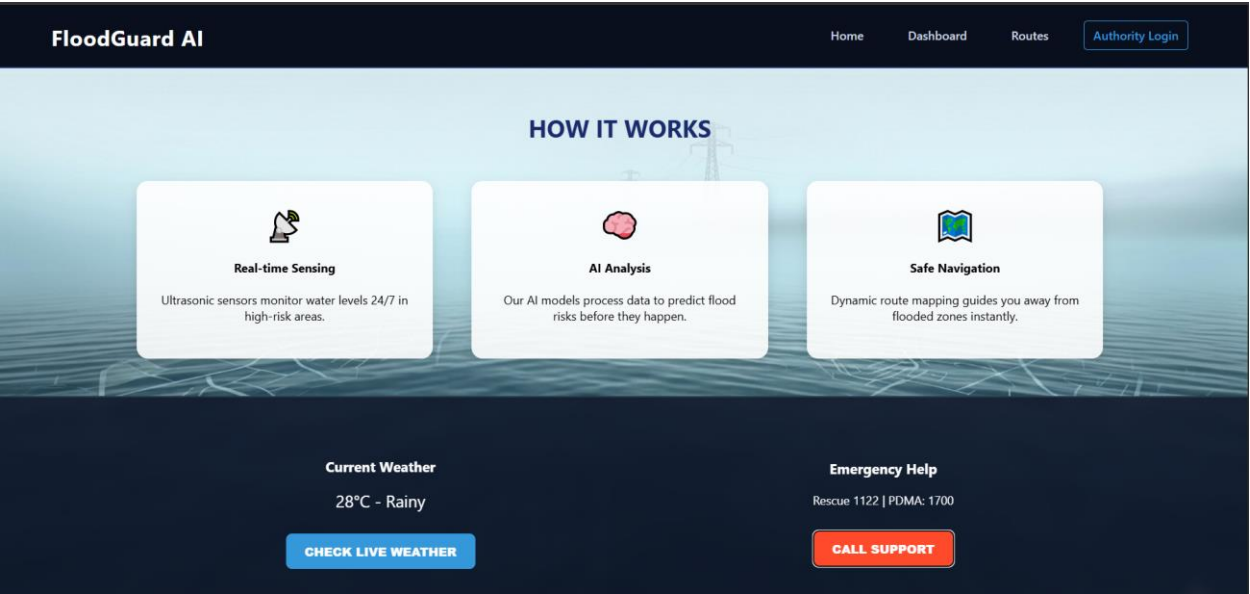
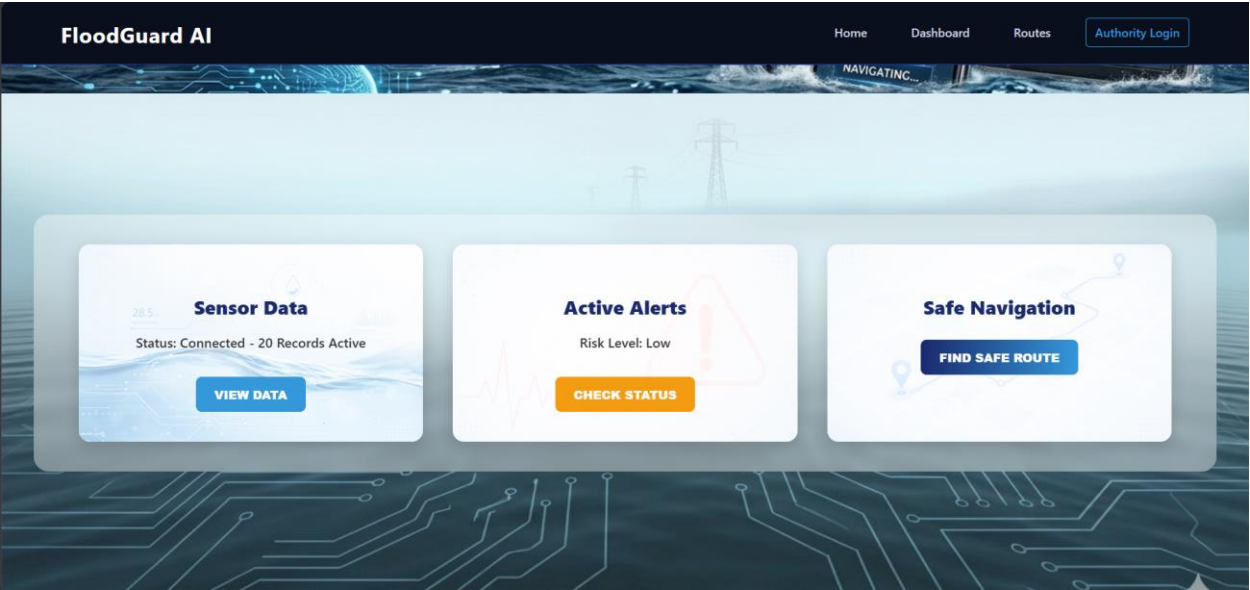
TOOLS & TECHNOLOGIES USED

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js
- Database: SQL Server

Server: Localhost (XAMPP / Node Server)

FRONTEND / WEBSITE UI

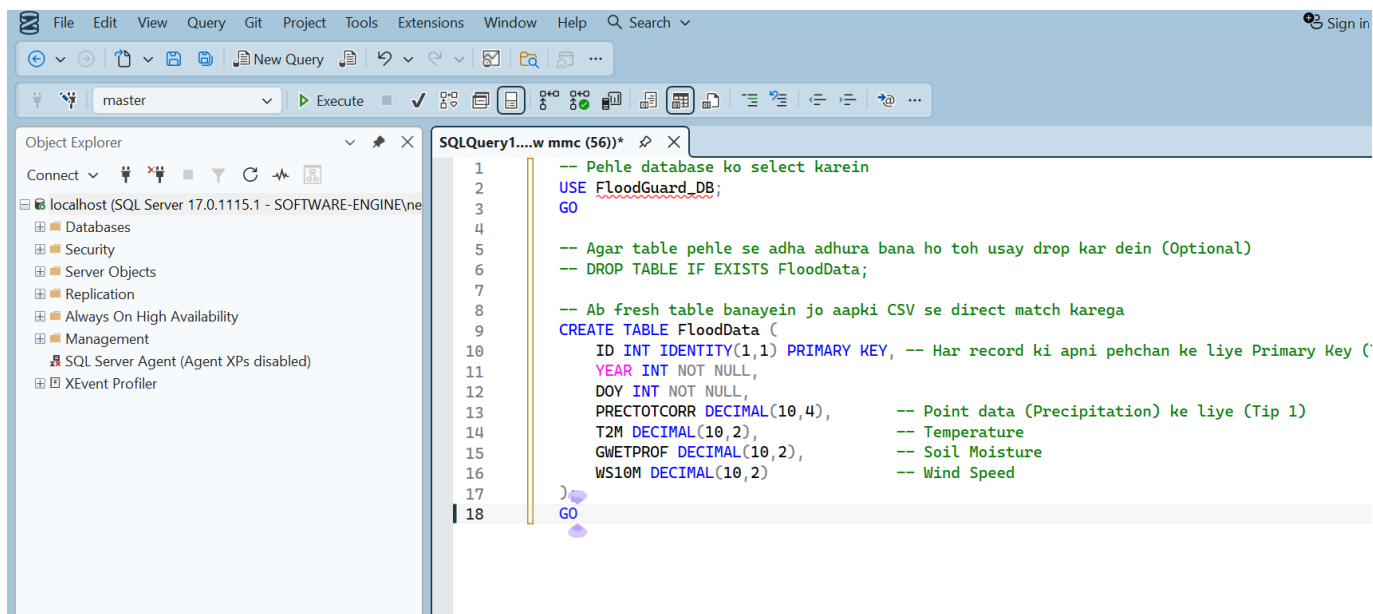




DATABASE SETUP



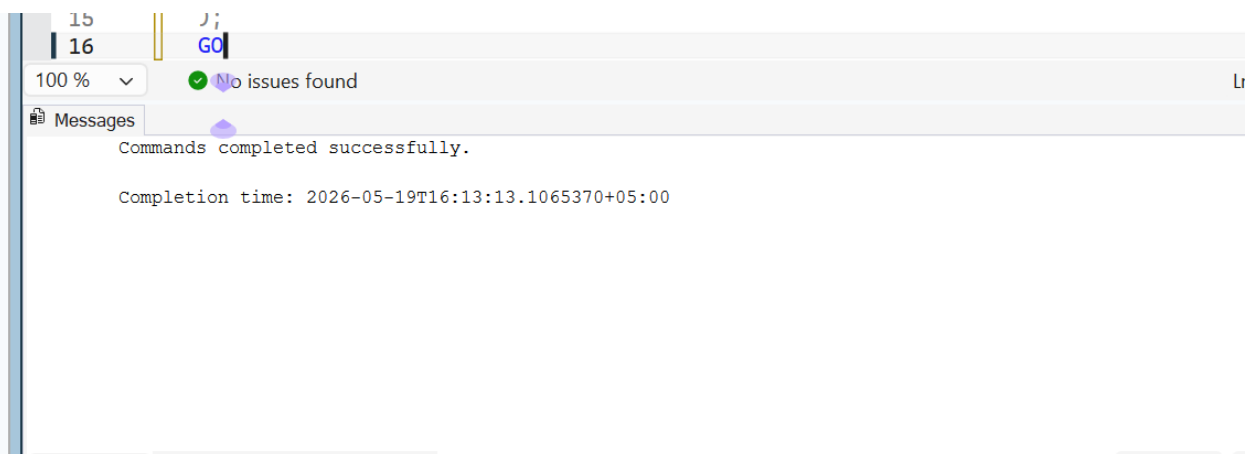
STRUCTURE OF DATABASE



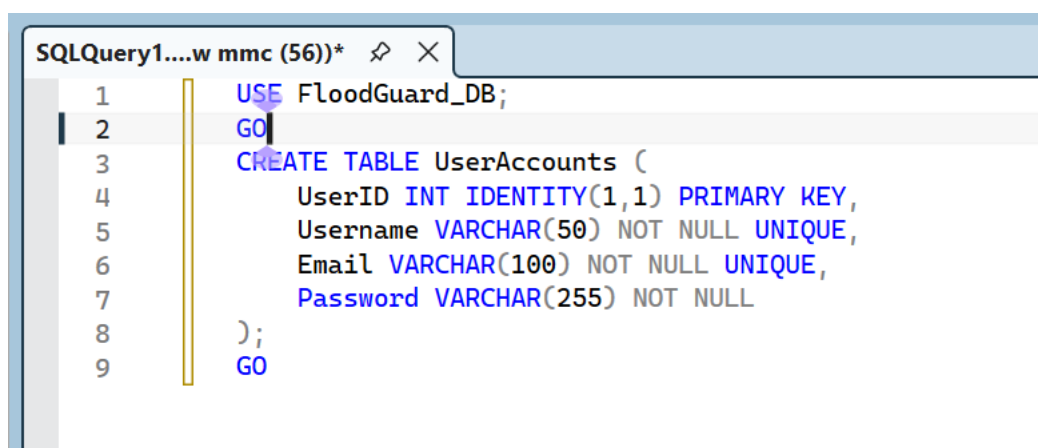
SETTING UP 51K DATA

YEAR	DOY	PRECOTCORR	T2M	GWETPROF	WS10M
1981	1	0	18.08	0.62	2.01
1981	2	0	18.33	0.62	2.62
1981	3	0	18.05	0.61	3.32
1981	4	0	18.7	0.61	2.94
1981	5	0	18.98	0.61	3.63
1981	6	2.25	17.51	0.61	3.68
1981	7	1.32	14.3	0.61	3.06
1981	8	0.47	12.63	0.61	2.93
1981	9	0	12.03	0.61	3.45
1981	10	0	12.79	0.61	3.24
1981	11	0	12.13	0.61	3.26

DATABASE CREATE SUCCESSFULLY

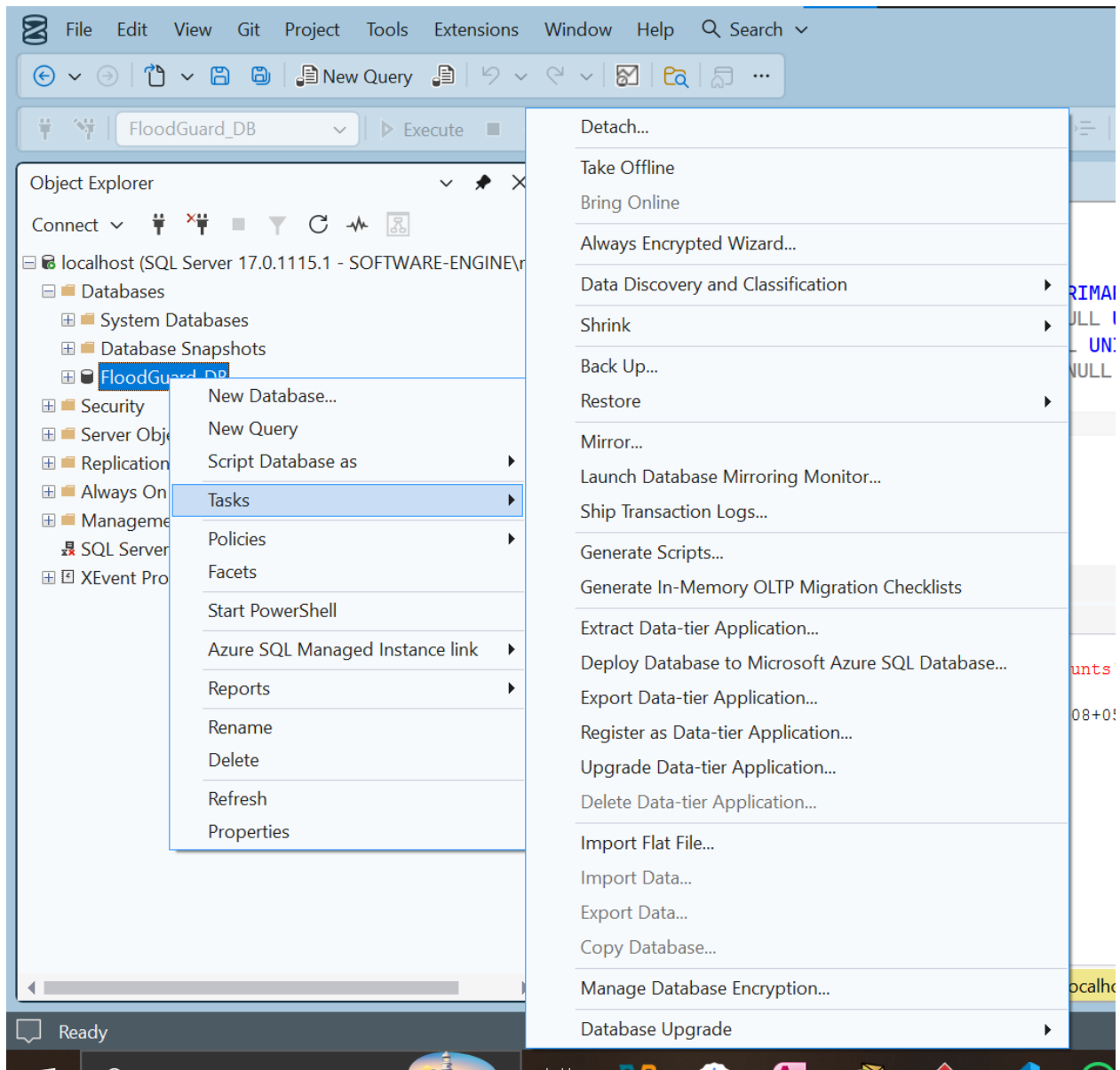


CREATE USER:



CONNECTION

- The system connects to the database and fetches data using **SQL queries**.
- Queries are used to access and display relevant flood data from the database.



SET LOCATION

- The **location** field is used to store the area or region of the sensor.
- It helps identify **where the flood data is coming from**.
- Example values:
 - Islamabad
 - Rawalpindi
 - River Side Area

Import Flat File 'FloodGuard_DB'

Specify Input File

Introduction
Specify Input File
Preview Data
Modify Columns
Summary
Results

Specify Input File
This operation will create a table from your input file.

Location of file to be imported
C:\Users\new mmc\OneDrive\Desktop\Flood_Dataset_51k_Records.csv Browse...

New table name:
FloodData

Table schema:
dbo

Help

PREVIEW DATA

- The **Preview Data** option is used to **view stored records in the table**.
- It helps in:
 - Checking if data is correctly inserted
 - Verifying sensor readings
 - Testing database queries

Import Flat File 'FloodGuard_DB'

Preview Data

Introduction
Specify Input File
Preview Data
Modify Columns
Summary
Results

Preview Data
This operation analyzed the input file structure to generate the preview below for up to the first 50 rows.

YEAR	DOY	PRECTOTCORR	T2M	GWETPROF	WS10M
1981	1	0.0	18.08	0.62	2.01
1981	2	0.0	18.33	0.62	2.62
1981	3	0.0	18.05	0.61	3.32
1981	4	0.0	18.7	0.61	2.94
1981	5	0.0	18.98	0.61	3.63
1981	6	2.25	17.51	0.61	3.68
1981	7	1.32	14.3	0.61	3.06
1981	8	0.47	12.63	0.61	2.93
1981	9	0.0	12.03	0.61	3.45
1981	10	0.0	12.79	0.61	3.24
1981	11	0.0	12.13	0.61	3.26
1981	12	0.0	11.47	0.61	3.45
1981	13	0.0	11.33	0.61	3.71
1981	14	0.0	12.71	0.61	3.89
1981	15	0.0	14.91	0.61	3.53
1981	16	0.0	17.49	0.61	2.89
1981	17	1.0	17.05	0.61	3.87

☒ Use Rich Data Type Detection - may provide a closer type fit. However, cells with anomalous values may be dropped.

< Previous Next > Cancel

MODIFY COLUMNS

- Columns can be modified to change:
 - Data type
 - Size
 - Constraints (NULL / NOT NULL)

WHY VARCHAR(MAX) IS USED?

- VARCHAR (MAX) is used to store **large text data**.

Reasons:

- It can store **variable-length data**
- Maximum size is up to **2GB**
- Useful when data length is **not fixed**

Import Flat File 'FloodGuard_DB'

Modify Columns

Introduction
Specify Input File
Preview Data
Modify Columns
Summary
[Results](#)

Modify Columns
This operation generated the following table schema. Please verify if schema is accurate, and if not, please make any changes.

Column Name	Data Type	Primary Key	Allow Nulls
YEAR	varchar(MAX)	☐	☒
DOY	varchar(MAX)	☐	☒
PRECTOTCORR	varchar(MAX)	☐	☒
T2M	varchar(MAX)	☐	☒
GWETPROF	varchar(MAX)	☐	☒
WS10M	varchar(MAX)	☐	☒

Row granularity of error reporting (performance impact with smaller ranges)

Summary of database:

Import Flat File 'FloodGuard_DB'

Summary

Introduction
Specify Input File
Preview Data
Modify Columns
Summary
Results

Summary
To complete the operation using the specified inputs, click Finish.

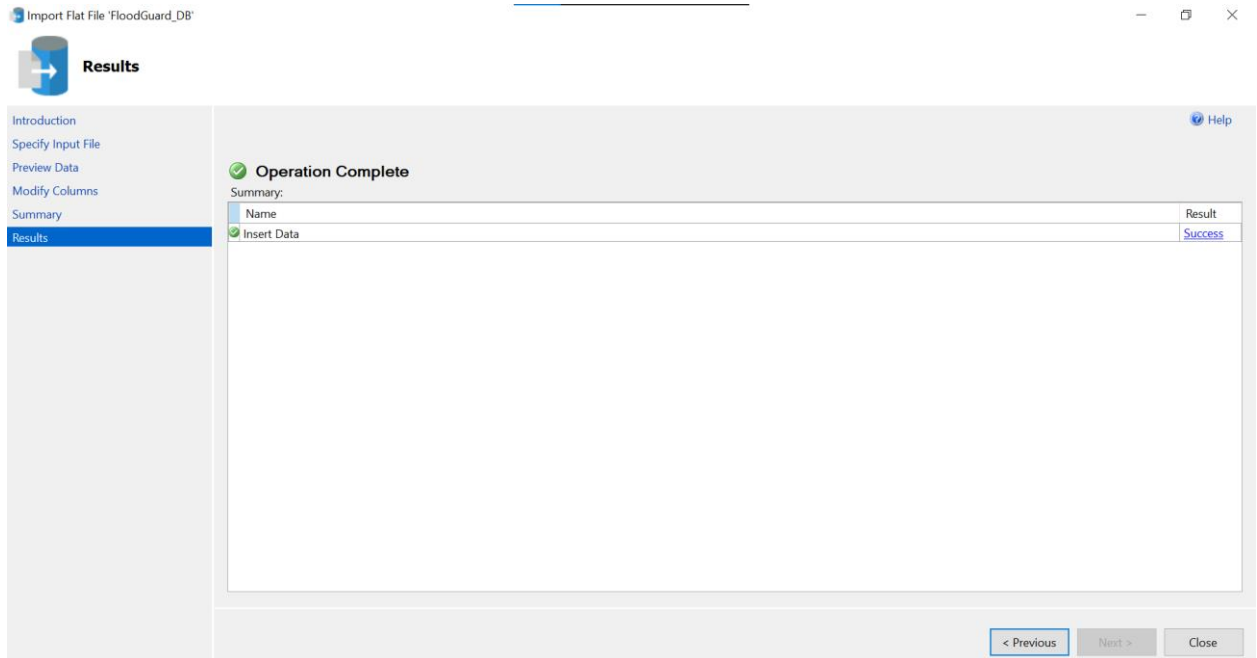
Import Information

- Name: Software-Engineer
- Database Name: FloodGuard_DB
- Table Name: dbo.FloodData
- File to be imported: C:\Users\new mmc\OneDrive\Desktop\Flood_Dataset_51k_Records.csv

< Previous Finish Cancel

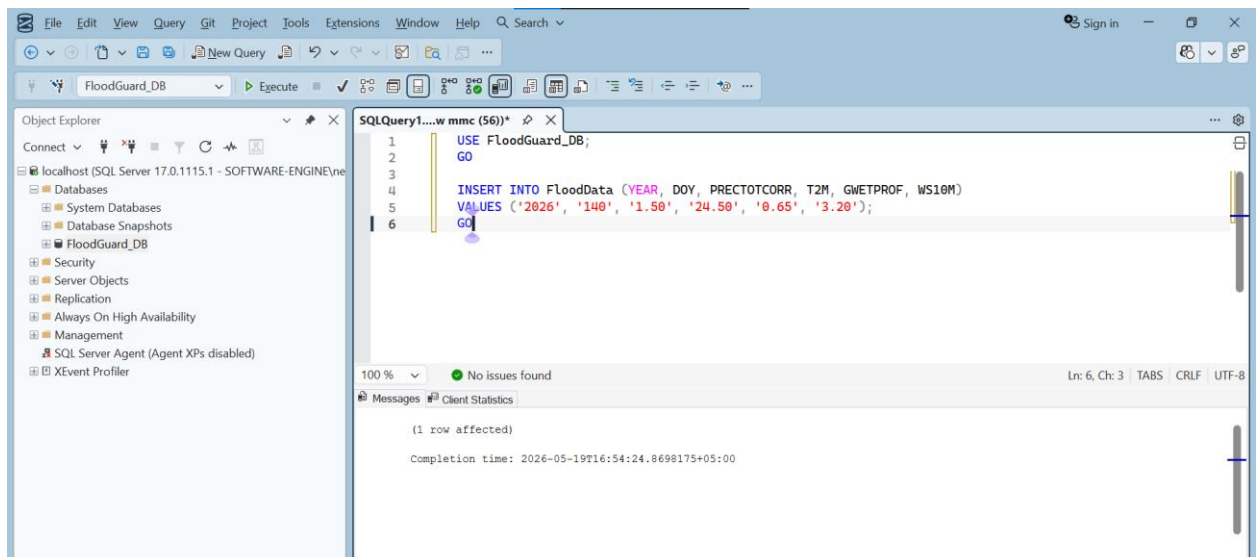
DATA INSERTION

- Test data was inserted into the database for system evaluation.
- Sample records were added to verify system functionality.



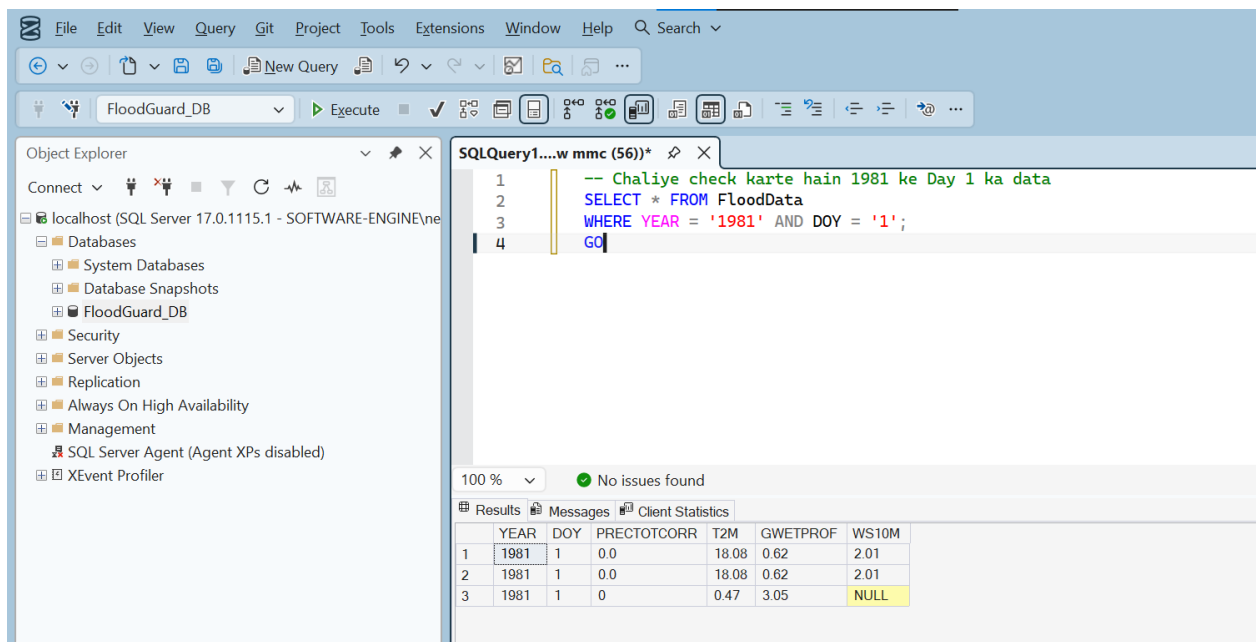
DATA TESTING

- Data was inserted specifically for testing purposes.
- This helped in checking system performance and accuracy.



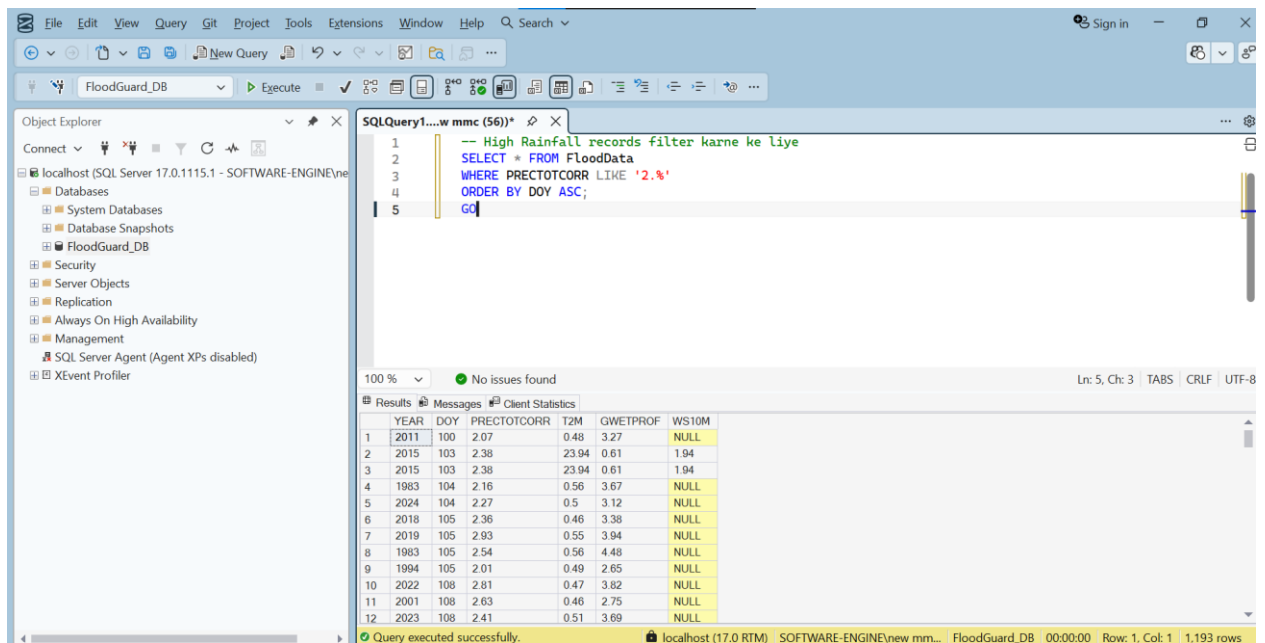
TIME-BASED SEARCH

- The system supports searching data based on a **specific time frame**.
- This allows users to analyze flood conditions at different times.



HIGH-RISK (DANGER LEVEL) FILTER

- A filter was implemented to identify **high-risk flood areas**.
- It highlights dangerous conditions for quick response.

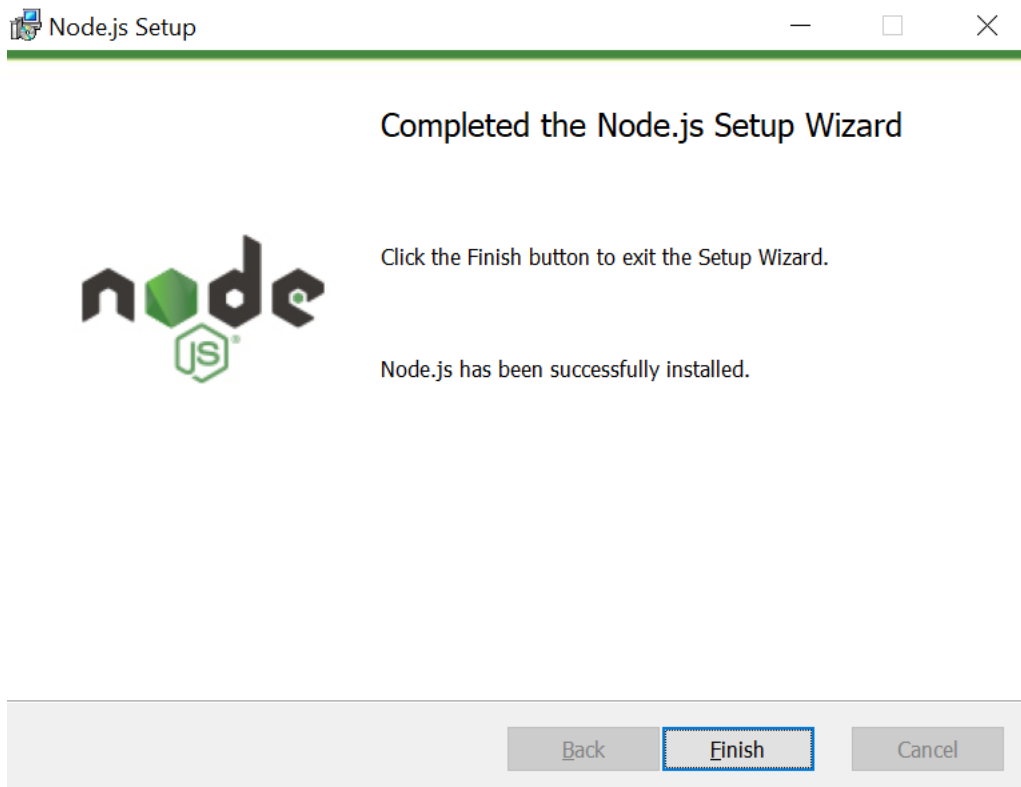
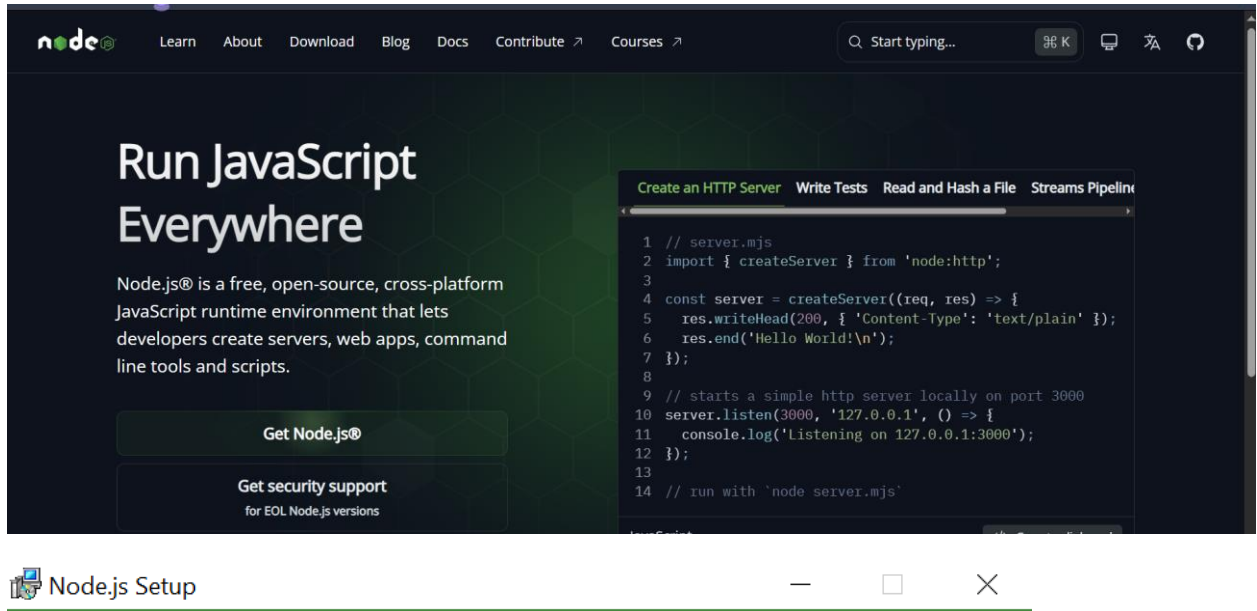


NODE.JS SETUP

- Node.js was used as a **backend runtime environment**.
- It acts as a **bridge between frontend (VS Code project) and SQL Server database**.

INSTALLATION & CONFIGURATION

- Node.js was installed to enable database connectivity.
- Required packages were installed using npm (Node Package Manager).



VERSION CHECK

- Node.js version was verified using CMD (Command Prompt).
- This ensures correct installation and compatibility.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\new mmc>node -v
v24.15.0

C:\Users\new mmc>
```

PACKAGE INSTALLATION

- Necessary dependencies were installed to run the backend server successfully.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\new mmc\Music\database website>npm init -y
Wrote to C:\Users\new mmc\Music\database website\package.json:

{
  "name": "database-website",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
C:\Users\new mmc\Music\database website>npm install express mssql msnodesqlv8 cors
npm warn deprecated prebuild-install@7.1.3: No longer maintained. Please contact the author o
f the relevant native addon; alternatives are available.

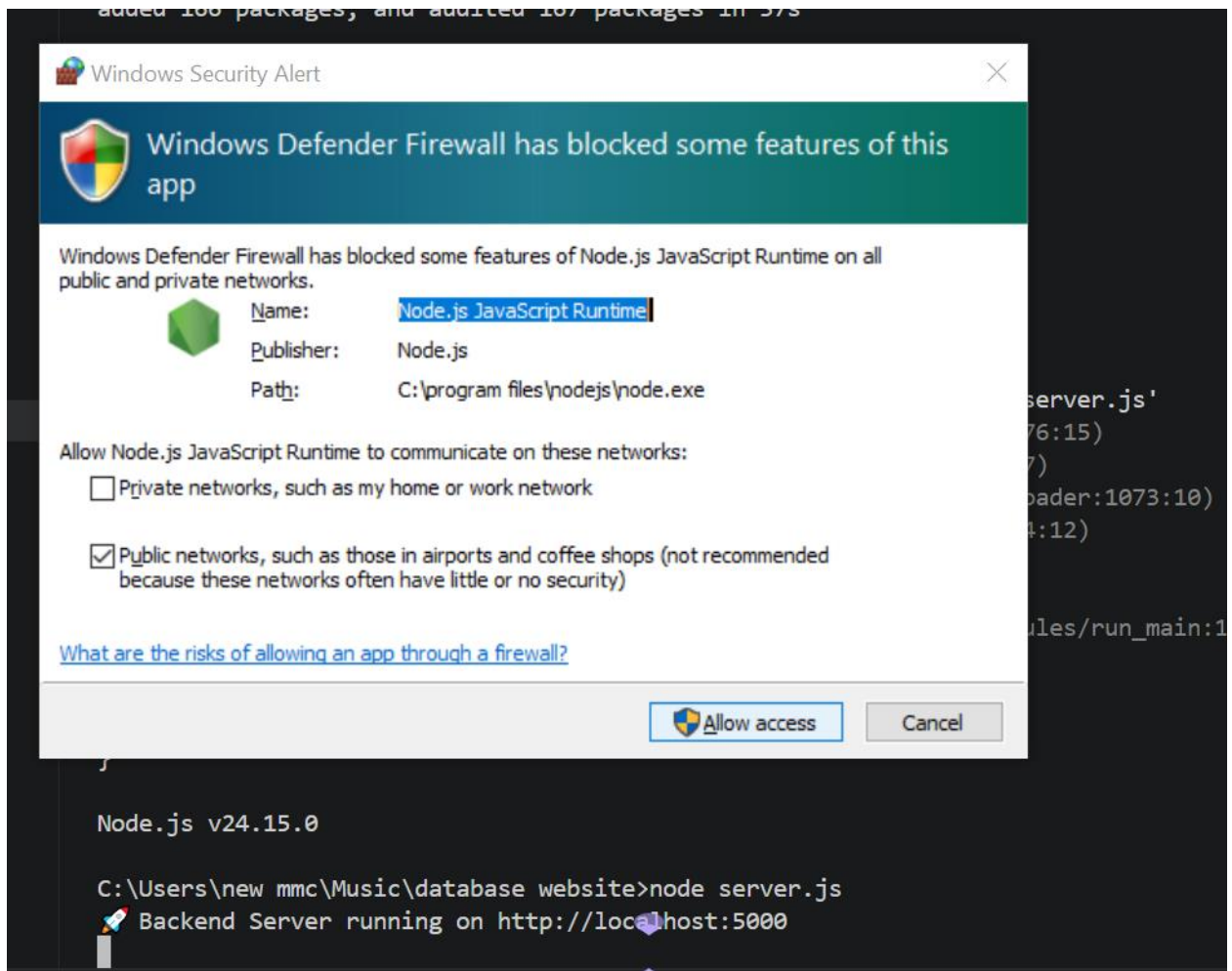
added 166 packages, and audited 167 packages in 37s

45 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

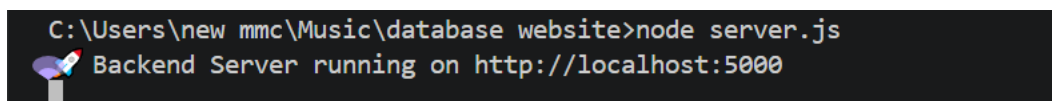
ACCESS PERMISSIONS

- Proper access permissions were configured to allow database connection.
- This ensures secure and authorized data communication.



BACKEND SERVER

- The backend server was successfully configured and running.
- It handles all API requests between frontend and database.

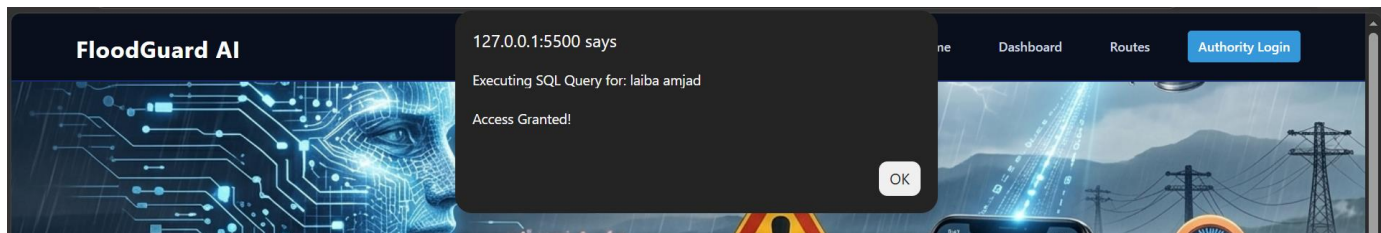


E. OUTPUT AND SCREENSHOTS

SQL QUERY TESTING

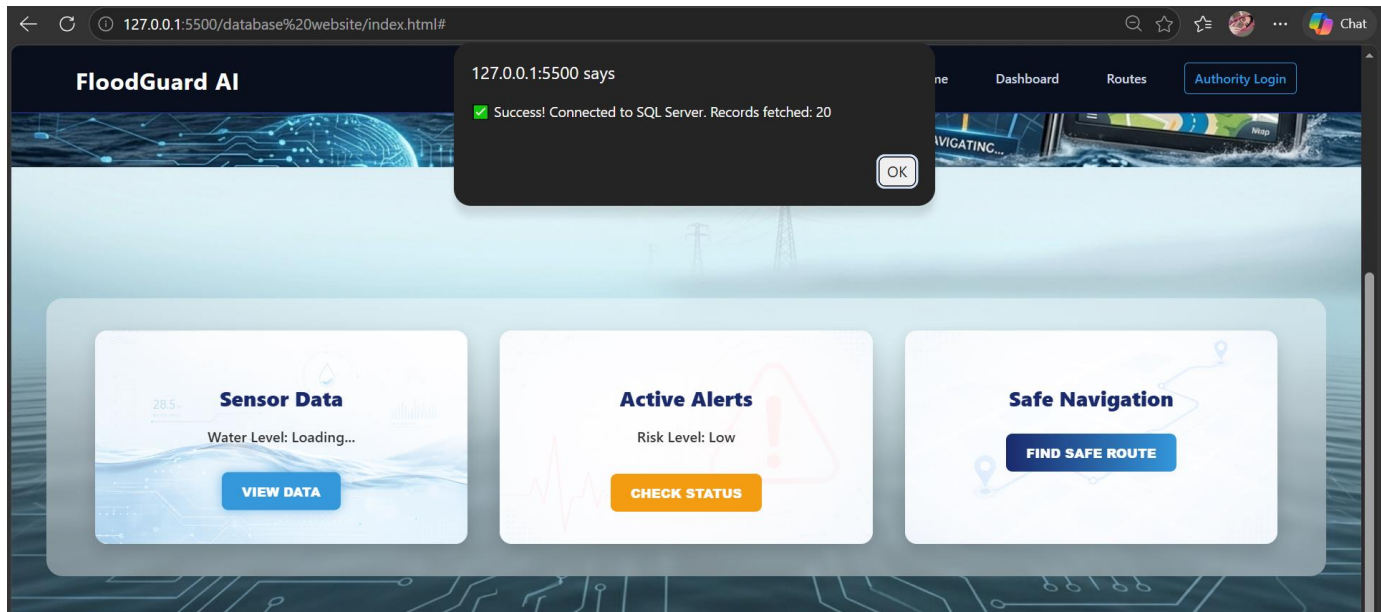
User Login Query:

Used to verify username and authentication.



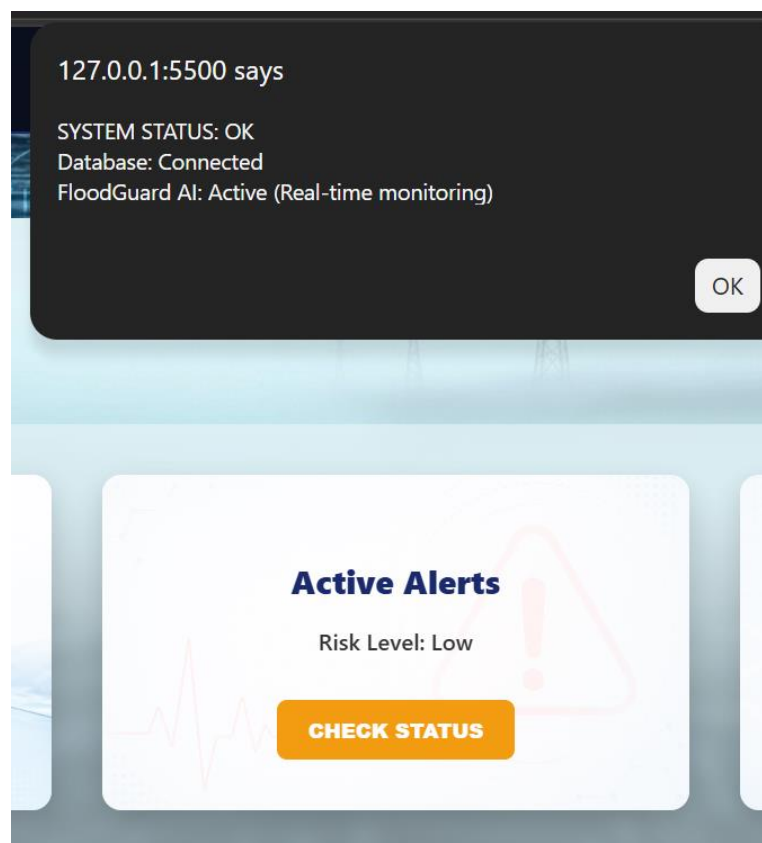
Sensor Data Query:

Retrieves live flood sensor data from database.



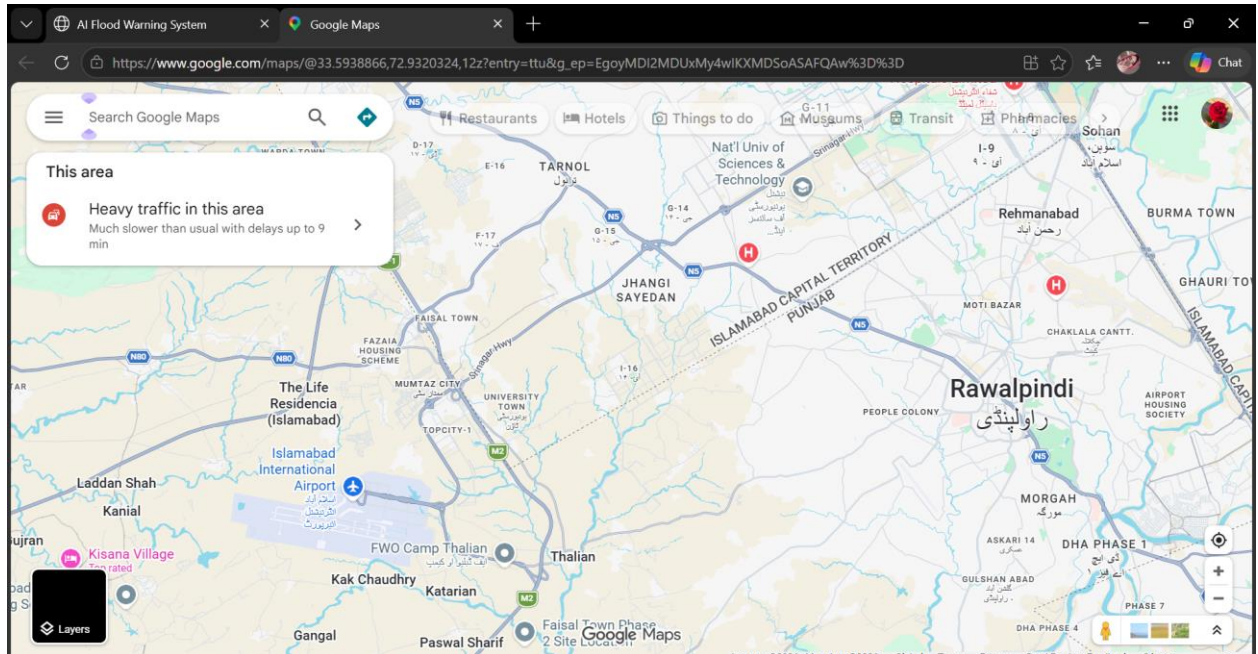
Status Check Query:

Shows current system or flood status.



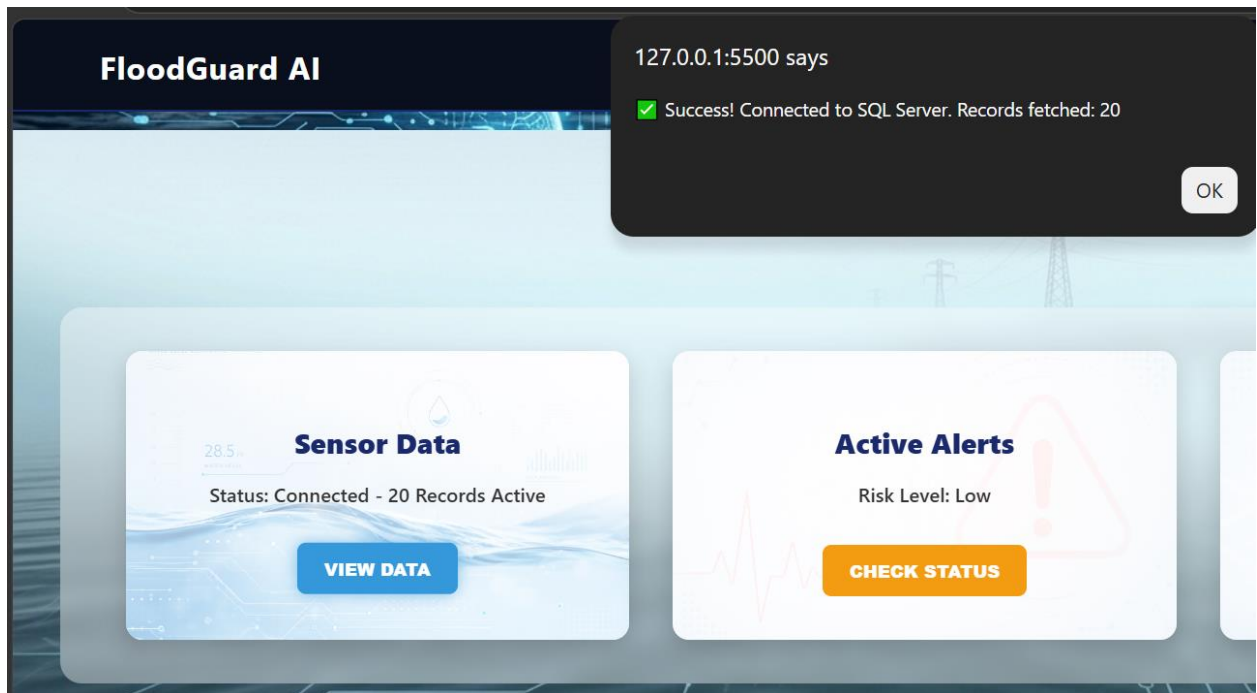
Safe Route Query:

Helps in finding the safest possible route during floods.



SYSTEM OUTPUTS

Sensor data is displayed in real-time



Live weather information is checked and shown



Emergency call feature is available for quick response

